EE 541 – Homework 5
Problem 1: Logistic Regression

Mengjia Shang
USC ID: 7338151449

Problem 1: Binary "2" Detector using Logistic Regression

I implemented a logistic regression classifier from scratch (using only NumPy) to detect the digit "2" in the MNIST dataset. The model computes $p(y = 1 \mid x) = \sigma(w^T x + b)$, trained with binary cross-entropy loss and gradient descent.

Question 1: Learning Rate Selection

I experimented with four different learning rates: $\alpha \in \{0.01, 0.1, 0.5, 1.0\}$, training each for 500 iterations and comparing the resulting loss curves.

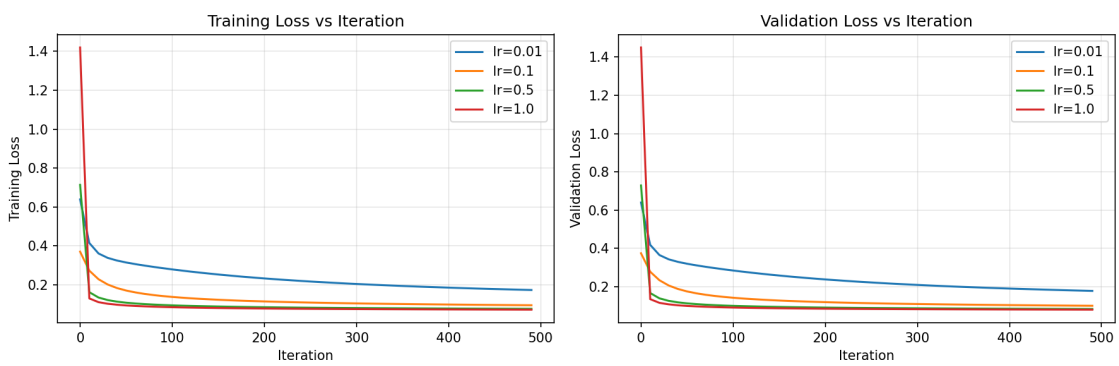

Figure 1: Comparison of training and validation loss for different learning rates.

I selected $\alpha = 0.5$ as the learning rate. With $\alpha = 0.01$, convergence was very slow — the loss was still decreasing significantly at 500 iterations, meaning it would need many more iterations to converge. With $\alpha = 0.1$, convergence was faster but still relatively slow. $\alpha = 0.5$ provided quick and stable convergence, with the loss settling into a plateau within a few hundred iterations. $\alpha = 1.0$ also converged quickly, but it showed slight instability in the early iterations, suggesting it was close to being too aggressive.

The reasoning here is essentially about balancing speed vs. stability: a large learning rate converges faster but risks overshooting, while a small one is more stable but wastes computation. $\alpha = 0.5$ hits a good sweet spot for this problem.

Question 2: Convergence Criteria

I used the **relative change in training loss** between consecutive evaluation points as my convergence criterion. Specifically, I considered the model converged when:

$$\frac{|L_t - L_{t-1}|}{|L_{t-1}|} < 10^{-5}$$

where L_t is the training loss at step t. I ran the model for 2000 iterations (evaluating every 10 iterations) and checked whether this criterion was met at the end. For all three models (no regularization, L2, and L1), the relative change in loss was well below 10^{-5} by the end of training, confirming convergence.

Additionally, looking at the learning curves (Figure 2), the loss curves for all models flatten out well before the 2000th iteration, providing visual confirmation that the models have converged.

Question 3: L1 vs. L2 Regularization Effects

I trained the model with both L1 and L2 regularization ($\lambda = 0.001$) and visualized the learned weight vectors as 28×28 images.

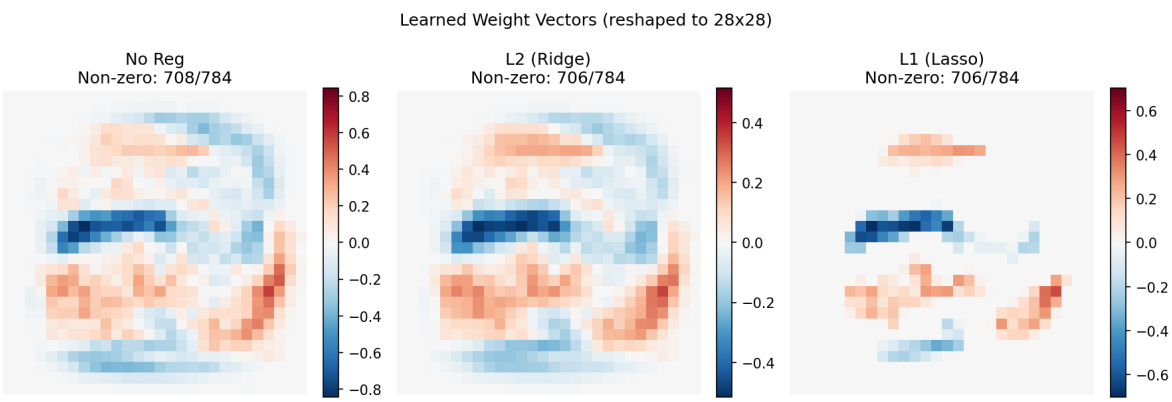

Figure 2: Weight vectors reshaped to 28×28 for the three models. Red = positive (evidence for "2"), blue = negative (evidence against "2").

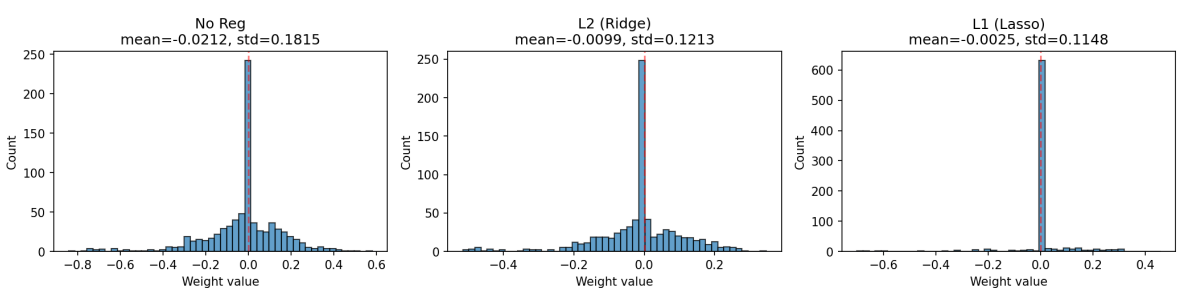

Figure 3: Distribution of weight values across the three models.

The key differences:

- **No regularization**: The weights are spread over a wide range. The weight image shows a noisy but recognizable pattern that roughly captures the shape of a "2."

- **L2 (Ridge)**: The weights are more uniform and smaller in magnitude. L2 penalizes the squared magnitude of weights, so it shrinks all weights toward zero without making any of them exactly zero. The weight image looks "smoother" than the unregularized version.
- **L1 (Lasso)**: Many weights are driven to exactly zero, making the model sparse. The non-zero weights are concentrated on the most informative pixels for detecting a "2." L1 effectively performs feature selection by zeroing out irrelevant pixels (like corner regions where a "2" would never appear).

This matches the theory: L2 distributes the penalty evenly, producing small but non-zero weights everywhere, while L1 encourages sparsity by pushing less important features to exactly zero.

Question 4: Learning Curves Analysis

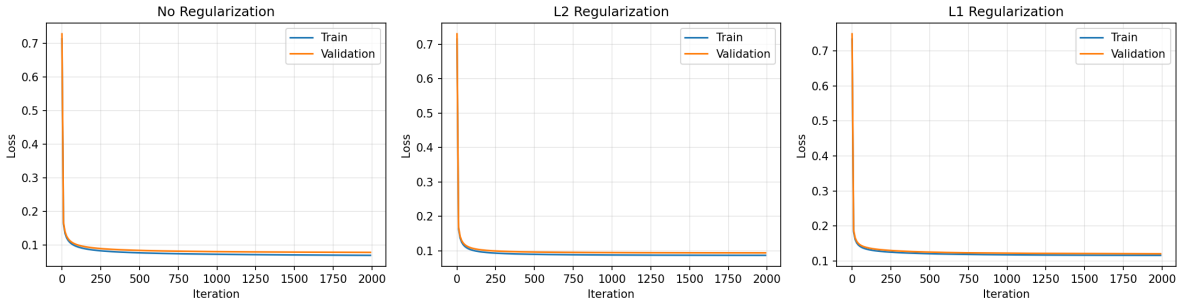

Figure 4: Training and validation loss vs. iteration for all three models.

The learning curves for the best model (L2 regularization) show several important things:

- Both training and validation loss decrease rapidly in the first ∼200 iterations and then plateau, confirming convergence.
- The training and validation loss curves are close together throughout training, which indicates the model is **not overfitting**. This is expected since logistic regression is a linear model with limited capacity — it doesn't have enough parameters to memorize the training data.
- The regularized models (L2 and L1) show slightly higher training loss but similar validation loss compared to the unregularized model. This is the expected effect of regularization: it constrains the model, which slightly hurts training performance but can improve generalization.
- The flat plateau at the end tells us that running for more iterations wouldn't help — the model has already extracted all the signal it can from the data (given its linear structure).

Question 5: Test Metrics and Class Imbalance

The accuracy for all models is around 97–98%, but this metric alone is misleading because of the **class imbalance**. Only about 10.3% of the test samples are digit "2" (1032 out of 10000). A naive classifier that always predicts "not 2" would achieve ∼90% accuracy without learning anything useful.

This is why we need the other metrics:

Model	Accuracy	Precision	Recall	F1
No Regularization	0.9810	0.9626	0.8488	0.9022
L2 ($\lambda = 0.001$)	0.9782	0.9678	0.8159	0.8854
L1 ($\lambda = 0.001$)	0.9756	0.9690	0.7888	0.8697

Table 1: Test set metrics for all three models.

- **Precision** ($\sim$0.97): When the model predicts "2," it's correct about 97% of the time. Very few false positives.
- **Recall** ($\sim$0.79–0.85): The model correctly identifies about 79–85% of actual "2"s. Some 2s are being missed (false negatives), likely cases where the handwriting style makes the "2" look unusual. Interestingly, L1 has the lowest recall, probably because zeroing out weights means losing some useful information.
- **F1 Score** ($\sim$0.87–0.90): The harmonic mean of precision and recall gives a more balanced picture. The unregularized model has the highest F1 of 0.90, while the L1 model is a bit lower at 0.87. This shows the model is doing a solid job despite the imbalance, though there's room for improvement on the recall side.

The relatively lower recall compared to precision is a common pattern in imbalanced classification tasks: the model becomes conservative and tends to predict the majority class when uncertain. To improve recall, one could adjust the decision threshold below 0.5, use class weights during training, or try oversampling the minority class.